

Best Programming Practice

1. Use Variables including for Fixed, User Inputs, and Results
2. Use Methods instead of writing code in the main() function
3. Proper naming conventions for all variables and methods
4. Proper Program Name and Class Name
5. Handle Checked and Unchecked Exceptions wherever possible
6. Proper Method Name which indicates action taking inputs and providing result

Sample Program 1: Create a program to find all the occurrences of a character in a string using charAt() method

- a. Take user input for the String and occurrences of the Character to find
- b. Write a method to find all the occurrences of the characters.
 - i. The logic used is to first find the number of occurrences of the character and
 - ii. then create an array to store the indexes of the character
- c. Call the method in the main and display the result

Java

```
// Program to find all the occurrences of a character in a string
import java.util.Scanner;
class StringAnalyzer {
    // Method to find all the index of a character in a string using charAt()
    // method and return them in an array
    public static int[] findAllIndexes(String text, char ch) {
        // The count is used to find the number of occurrences of the character
        int count = 0;
        for (int i = 0; i < text.length(); i++) {
            if (text.charAt(i) == ch) {
                count++;
            }
        }

        // Create an array to store the indexes of the character
        int[] indexes = new int[count];
        int j = 0;
        for (int i = 0; i < text.length(); i++) {
            if (text.charAt(i) == ch) {
                indexes[j] = i;
                j++;
            }
        }
        return indexes;
    }
}
```

```
public static void main(String[] args) {  
    // Take user input for Text and Character to check Occurrences  
    Scanner sc = new Scanner(System.in);  
    System.out.print(Enter a text: ");  
    String text = sc.nextLine();  
    System.out.print("Enter a character to find the occurrences: ");  
    char ch = sc.next().charAt(0);  
  
    // Find the occurrences of the character  
    int[] indexes = findAllIndexes(text, ch);  
  
    // Display the occurrences of the character  
    System.out.println("Indexes of the character '" + ch + "': ");  
    for (int i = 0; i < indexes.length; i++) {  
        System.out.print(indexes[i] + " ");  
    }  
}
```

Level 2 Practice Programs

1. Write a program to find and return the length of a string without using the `length()` method

Hint =>

- a. Take user input using the `Scanner next()` method
- b. Create a method to find and return a string's length without using the built-in `length()` method. The logic for this is to use the infinite loop to count each character till the `charAt()` method throws a runtime exception, handles the exception, and then return the count
- c. The main function calls the user-defined method as well as the built-in `length()` method and displays the result

```
import java.util.Scanner;
public class StringLength {
    public static void main(String[] args) {

        //creating scanner object to take input
        Scanner input = new Scanner(System.in);

        //taking user input for a string
        System.out.print("Enter a string : ");
        String str = input.next();

        //creating a variable lengthByMethod to indicate length of
        string by method
        int lengthByMethod = findLength(str);

        //printing details
        System.out.println("Length of string " + str + " is " +
lengthByMethod + " without length() method");
        System.out.println("Length of string " + str + " is " +
str.length() + " by using length() method");

        //closing the input
        input.close();
    }
    //method to find length of given string
    public static int findLength(String str) {
```

```
//finding the length of string and storing it in len variable
int len = 0;
try {
    while(true) {
        str.charAt(len);
        len++;
    }
} catch (Exception e) {
    //do nothing if string out of index exception arises
}

//returning the length
return len;
}
}
```

2. Write a program to split the text into words, compare the result with the split() method and display the result

Hint =>

- a. Take user input using the **Scanner nextLine()** method
- b. Create a Method to find the length of the String without using the built-in length() method.
- c. Create a Method to split the text into words using the charAt() method without using the String built-in **split()** method and return the words. Use the following logic
 - i. Firstly Count the number of words in the text and create an array to store the indexes of the spaces for each word in a 1D array
 - ii. Then Create an array to store the words and use the indexes to extract the words
- d. Create a method to compare the two String arrays and return a boolean
- e. The main function calls the user-defined method and the built-in **split()** method. Call the user defined method to compare the two string arrays and display the result

```
import java.util.*;
class WordSplitter {
    public static void main(String[] args) {

        //Taking input from user
        Scanner input = new Scanner(System.in);
        System.out.print("Enter the sentence : ");
```

```
String sentence = input.nextLine();

//printing the length of sentence
System.out.println("Length of your sentence is " +
findLength(sentence));

//creating variable wordCount to store count of words
int wordCount = wordCounter(sentence);
//printing the word count
System.out.println("Number of words = " + wordCount);

//calling the splitter method to get array of words
String strArray = Arrays.toString(splitter(sentence));
//printing the array of words and storing it in strArray string
after converting it to string
System.out.println("The array is " + strArray);

//printing array of words by builtin split() method and storing
it in strArray2 string after converting it to string
String strArray2 = Arrays.toString(sentence.split(" "));
System.out.println("The array is " + strArray2);

//comparing both arrays (array by method and array by builtin
split() method)
System.out.println("Both arrays are same? " +
compareStringArrays(splitter(sentence), sentence.split(" ")));
}

//method to find length of given string
public static int findLength(String str) {

//finding the length of string and storing it in len variable
int len = 0;
try {
    while(true) {
        str.charAt(len);
        len++;
    }
} catch (Exception e) {
    //do nothing if string out of index exception arises
}
```

```

        //returning the length
        return len;
    }
    //method to count words in a sentence
    public static int wordCounter(String sentence) {

        //determining the length of string
        int len = findLength(sentence);
        //creating the wordcount variable to indicate the count of
words in the sentence
        int wordcount = 1;
        for(int i=0;i<len;i++){
            if(sentence.charAt(i)==' '){
                wordcount++;
            }
        }
        //returning the counter
        return wordcount;
    }

    //method to split the string
    public static String[] splitter(String sentence) {

        //creating variable wordCount to store count of words
        int wordCount = wordCounter(sentence);

        int index = 0;
        String[] strArray = new String[wordCount];

        //variables to indicate starting and ending index of word
        int low = 0;
        int high = 0;

        for(int i = 0;i<sentence.length();i++) {
            if(sentence.charAt(i) != ' '){
                high++;
            }else{
                //storing the word into the array
                strArray[index] = sentence.substring(low,high);
            }
        }
    }

```

```

        index++;
        high = high + 1;
        low = high;
    }
}

//inserting the last word
strArray[index] = sentence.substring(low,high);

//returning the array of words
return strArray;
}

//method to compare two string arrays
public static boolean compareStringArrays(String array1[], String
array2[]) {

    //returning false if length are different
    if(array1.length != array2.length) {
        return false;
    }

    //returning false if elements differ
    for(int i=0; i<array1.length; i++) {
        if (!(array1[i].equals(array2[i]))) {
            return false;
        }
    }
    //returning true because arrays are same
    return true;
}
}

```

3. Write a program to split the text into words and return the words along with their lengths in a 2D array

Hint =>

- a. Take user input using the **Scanner nextLine()** method
- b. Create a Method to split the text into words using the charAt() method without using the String built-in **split()** method and return the words.
- c. Create a method to find and return a string's length without using the length() method.

- d. Create a method to take the word array and return a 2D String array of the word and its corresponding length. Use String built-in function String.valueOf() to generate the String value for the number
- e. The main function calls the user-defined method and displays the result in a tabular format. During display make sure to convert the length value from String to Integer and then display

```
import java.util.*;
class WordSplitter2D {
    public static void main(String[] args) {

        //Taking input from user
        Scanner input = new Scanner(System.in);
        System.out.print("Enter the sentence : ");
        String sentence = input.nextLine();

        //printing the length of sentence
        System.out.println("Length of your sentence is " +
findLength(sentence));

        //creating variable wordCount to store count of words
        int wordCount = wordCounter(sentence);
        //printing the word count
        System.out.println("Number of words = " + wordCount);

        //calling the splitter method to get array of words
        String strArray = Arrays.toString(splitter(sentence));
        //printing the array of words and storing it in strArray string
after converting it to string
        System.out.println("The array is " + strArray);

        //printing array of words by builtin split() method and storing
it in strArray2 string after converting it to string
        String strArray2 = Arrays.toString(sentence.split(" "));
        System.out.println("The array is " + strArray2);

        //print the 2-D array
        String[][] strLenArray = findstrLenArray(sentence);
        System.out.println("word    length" );
        System.out.println("-----" );

        for(int i=0;i<strLenArray.length;i++) {
```



```

        for(int j=0;j<strLenArray[0].length;j++){

            System.out.print(strLenArray[i][j] + "      ");
        }
        System.out.println();
    }

}

//method to find length of given string
public static int findLength(String str) {

    //finding the length of string and storing it in len variable
    int len = 0;
    try {
        while(true) {
            str.charAt(len);
            len++;
        }
    } catch (Exception e) {
        //do nothing if string out of index exception arises
    }

    //returning the length
    return len;
}

//method to count words in a sentence
public static int wordCounter(String sentence) {

    //creating the wordcount variable to indicate the count of
words in the sentence
    int wordcount = 1;
    for(int i=0;i<sentence.length();i++){
        if(sentence.charAt(i)==' '){
            wordcount++;
        }
    }

    //returning the counter
    return wordcount;
}

//method to split the string

```

```

public static String[] splitter(String sentence) {

    //creating variable wordCount to store count of words
    int wordCount = wordCounter(sentence);

    int index = 0;
    String[] strArray = new String[wordCount];

    //variables to indicate starting and ending index of word
    int low = 0;
    int high = 0;

    for(int i = 0; i < sentence.length(); i++) {
        if(sentence.charAt(i) != ' '){
            high++;

        }else{
            //storing the word into the array
            strArray[index] = sentence.substring(low, high);
            index++;
            high = high + 1;
            low = high;
        }
    }

    //inserting the last word
    strArray[index] = sentence.substring(low, high);

    //returning the array of words
    return strArray;
}

//method to return a 2d string array of words
public static String[][] findstrLenArray(String sentence) {

    //calling the splitter method to split the sentence in an 1d
    array

    String[] strArray = splitter(sentence);

    //Making the 2-D array strLenArray to store the words of
    sentence along with the lengths [word ,length]
    String[][] strLenArray = new String[strArray.length][2];
    for(int i=0; i < strArray.length; i++){

```

```

        strLenArray[i][0] = strArray[i];
        strLenArray[i][1] = Integer.toString(findLength(strArray[i]));
    }

    //returning the 2d array
    return strLenArray;
}
}

```

4. Write a program to split the text into words and find the shortest and longest strings in a given text

Hint =>

- Take user input using the **Scanner** *nextLine()* method
- Create a Method to split the text into words using the *charAt()* method without using the String built-in **split()** method and return the words.
- Create a method to find and return a string's length without using the *length()* method.
- Create a method to take the word array and return a 2D String array of the word and its corresponding length. Use String built-in function *String.valueOf()* to generate the String value for the number
- Create a Method that takes the 2D array of word and corresponding length as parameters, find the shortest and longest string and return them in an 1D int array.
- The main function calls the user-defined methods and displays the result.

```

import java.util.*;
class WordSplitterShortestAndLongest {
    public static void main(String[] args) {

        //Taking input from user
        Scanner input = new Scanner(System.in);
        System.out.print("Enter the sentence : ");
        String sentence = input.nextLine();

        //printing the length of sentence
        System.out.println("Length of your sentence is " +
findLength(sentence));

        //creating variable wordCount to store count of words
        int wordCount = wordCounter(sentence);
        //printing the word count
        System.out.println("Number of words = " + wordCount);
    }
}

```

```

        //calling the splitter method to get array of words
        String strArray = Arrays.toString(splitter(sentence));
        //printing the array of words and storing it in strArray string
        after converting it to string
        System.out.println("The array is " + strArray);

        //printing array of words by builtin split() method and storing
        it in strArray2 string after converting it to string
        String strArray2 = Arrays.toString(sentence.split(" "));
        System.out.println("The array is " + strArray2);

        //print the 2-D array
        String[][] strLenArray = findstrLenArray(sentence);
        System.out.println("word    length" );
        System.out.println("-----" );

        for(int i=0;i<strLenArray.length;i++) {
            for(int j=0;j<strLenArray[0].length;j++){

                System.out.print(strLenArray[i][j] + "    ");
            }
            System.out.println();
        }

        //creating a array indexes to store indexes of shortest and
        longest string
        int[] indexes = shortestAndLongestString(strLenArray);

        //printing the shortest and longest strings
        System.out.println("Shortest string is " +
        strLenArray[indexes[0]][0]);
        System.out.println("Longest string is " +
        strLenArray[indexes[1]][0]);
    }

    //method to find length of given string
    public static int findLength(String str) {

        //finding the length of string and storing it in len variable
        int len = 0;
        try {
            while(true) {

```

```

        str.charAt(len);
        len++;
    }
} catch (Exception e) {
    //do nothing if string out of index exception arises
}

//returning the length
return len;
}
//method to count words in a sentence
public static int wordCounter(String sentence) {

    //creating the wordcount variable to indicate the count of
words in the sentence
    int wordcount = 1;
    for(int i=0;i<sentence.length();i++){
        if(sentence.charAt(i)==' '){
            wordcount++;
        }
    }

    //returning the counter
    return wordcount;
}

//method to split the string
public static String[] splitter(String sentence) {

    //creating variable wordCount to store count of words
    int wordCount = wordCounter(sentence);

    int index = 0;
    String[] strArray = new String[wordCount];

    //variables to indicate starting and ending index of word
    int low = 0;
    int high = 0;

    for(int i = 0;i<sentence.length();i++) {
        if(sentence.charAt(i) != ' '){
            high++;

```

```

        }else{
            //storing the word into the array
            strArray[index] = sentence.substring(low,high);
            index++;
            high = high + 1;
        }
        low = high;
    }
}

//inserting the last word
strArray[index] = sentence.substring(low,high);

//returning the array of words
return strArray;
}

//method to return a 2d string array of words
public static String[][] findstrLenArray(String sentence) {

    //calling the splitter method to split the sentence in an 1d
    array
    String[] strArray = splitter(sentence);

    //Making the 2-D array strLenArray to store the words of
    sentence along with the lengths [word ,length]
    String[][] strLenArray = new String[strArray.length][2];
    for(int i=0; i<strArray.length; i++){
        strLenArray[i][0] = strArray[i];
        strLenArray[i][1] = Integer.toString(findLength(strArray[i]));
    }

    //returning the 2d array
    return strLenArray;
}

//method to find shortest and longest strings
public static int[] shortestAndLongestString(String[][] strLenArray)
{

    //creating variable minIndex and maxIndex to indicate indexes
    of minimum and maximum elements
    int minIndex = 0;
    int maxIndex = 0;
    //loop through the array for finding minimum and maximum

```

```

indexes
    for(int i=0; i<strLenArray.length; i++) {
        if(Integer.parseInt(strLenArray[i][1]) <
Integer.parseInt(strLenArray[minIndex][1])) {
            minIndex = i;
        }
        if(Integer.parseInt(strLenArray[i][1]) >
Integer.parseInt(strLenArray[maxIndex][1])) {
            minIndex = i;
        }
    }
    //storing indexes shortest and longest string in an array
{shortest, longest}
    int[] indexes = {minIndex, maxIndex};
    //returning the array
    return indexes;
}
}

```

5. Write a program to find vowels and consonants in a string and display the count of Vowels and Consonants in the string

Hint =>

- Create a method to check if the character is a vowel or consonant and return the result. The logic used here is as follows:
 - Convert the character to lowercase if it is an uppercase letter using the ASCII values of the characters
 - Check if the character is a vowel or consonant and return Vowel, Consonant, or Not a Letter
- Create a Method to Method to find vowels and consonants in a string using charAt() method and finally return the count of vowels and consonants in an array
- Finally, the main function takes user inputs, calls the user-defined methods, and displays the result.

```

import java.util.Scanner;
public class VowelConsonantCounter2 {
    public static void main(String[] args) {

        //creating scanner object to take input
        Scanner input = new Scanner(System.in);
    }
}

```

```
//taking user input for a string
System.out.print("Enter a string : ");
String str = input.nextLine();

//creating variable vowel to store count of vowels
int vowel = countVowel(str);
//creating variable consonant to store count of vowels
int consonant = countConsonant(str);

//printing the results
System.out.println("Number of vowels are " + vowel);
System.out.println("Number of consonants are " + consonant);

//creating array status to store status (vowel, consonant or
invalid) of string
String[] status = checkString(str);

//printing whether a character is vowel, consonant or not a
character
for(int i=0; i<str.length(); i++) {
    System.out.println("Character " + str.charAt(i) + " is "
+ status[i]);
}

//closing the input
input.close();
}

// Method to convert the string to lowercase using ASCII values
public static String convertToLower(String text) {
    StringBuilder result = new StringBuilder();

    // Loop through each character in the input text
    for (int i = 0; i < text.length(); i++) {
        char ch = text.charAt(i);

        // Check if the character is uppercase
        if (ch >= 'A' && ch <= 'Z') {
            // Converting it to lowercase by adjusting the ASCII value
            ch = (char) (ch + 32);
        }
    }
}
```



```

        // Adding the character to the result
        result.append(ch);
    }
    // Returning the converted string
    return result.toString();
}

//method to check if a number is vowel or consonant
public static String[] checkString(String str) {
    //converting string in lowercase
    str = convertToLower(str);

    //creating array status to store status (vowel, consonant or
invalid) of string
    String[] status = new String[str.length()];

    //checking each character of string
    for(int i=0; i<str.length(); i++) {
        //checking for invalid character
        if((int)str.charAt(i) < 65 || ((int)str.charAt(i) > 91 &&
(int)str.charAt(i) < 97) || (int)str.charAt(i) > 123) {
            status[i] = "not a character";
        }else if(str.charAt(i) == 'a' || str.charAt(i) == 'e' ||
str.charAt(i) == 'i' || str.charAt(i) == 'o' || str.charAt(i) == 'u') {
            status[i] = "vowel"; //checking for vowel
        } else {
            status[i] = "consonant"; //checking for consonant
        }
    }
    //returning the status array
    return status;
}

//method to count vowels
public static int countVowel(String str) {

    //converting string in lowercase
    str = convertToLower(str);

    //creating variable count to indicate count of vowels

```

```

        int count = 0;
        for(int i=0; i<str.length(); i++) {
            if(str.charAt(i) == 'a' || str.charAt(i) == 'e' ||
str.charAt(i) == 'i' || str.charAt(i) == 'o' || str.charAt(i) == 'u') {
                count++;
            }
        }
        //returning the count
        return count;
    }
    //method to count consonant
    public static int countConsonant(String str) {
        //creating variable count to indicate count of consonants
        int count = 0;
        for(int i=0; i<str.length(); i++) {
            //incrementing the count if current character is not a
vowel
            if(!(str.charAt(i) == 'a' || str.charAt(i) == 'e' ||
str.charAt(i) == 'i' || str.charAt(i) == 'o' || str.charAt(i) == 'u')) {
                count++;
            }
        }
        //returning the count
        return count;
    }
}

```

6. Write a program to find vowels and consonants in a string and display the character type - Vowel, Consonant, or Not a Letter

Hint =>

- Create a method to check if the character is a vowel or consonant and return the result. The logic used here is as follows:
 - Convert the character to lowercase if it is an uppercase letter using the ASCII values of the characters
 - Check if the character is a vowel or consonant and return Vowel, Consonant, or Not a Letter
- Create a Method to find vowels and consonants in a string using charAt() method and return the character and vowel or consonant in a 2D array
- Create a Method to display the 2D Array of Strings in a Tabular Format

- d. Finally, the main function takes user inputs, calls the user-defined methods, and displays the result.

```
import java.util.Scanner;
public class VowelConsonantChecker {
    public static void main(String[] args) {

        //creating scanner object to take input
        Scanner input = new Scanner(System.in);

        //taking user input for a string
        System.out.print("Enter a string : ");
        String str = input.nextLine();

        //creating array status to store status (vowel, consonant or
        invalid) of string
        String[][] status = checkString(str);

        //printing whether a character is vowel, consonant of not a
        character
        for(int i=0; i<str.length(); i++) {
            System.out.println("Character " + status[i][0] + " is " +
            status[i][1]);
        }

        //closing the input
        input.close();
    }

    // Method to convert the string to lowercase using ASCII values
    public static String convertToLower(String text) {
        StringBuilder result = new StringBuilder();

        // Loop through each character in the input text
        for (int i = 0; i < text.length(); i++) {
            char ch = text.charAt(i);

            // Check if the character is uppercase
            if (ch >= 'A' && ch <= 'Z') {
                // Converting it to lowercase by adjusting the ASCII value
                ch = (char) (ch + 32);
            }

            // Adding the character to the result
        }
    }
}
```

```

        result.append(ch);
    }
    // Returning the converted string
    return result.toString();
}
//method to check if a number is vowel or consonant
public static String[][] checkString(String str) {
    //converting string in lowercase
    str = convertToLower(str);

    //creating 2d array status to store status (vowel, consonant or
invalid) of string
    String[][] status = new String[str.length()][2];

    //checking each character of string
    for(int i=0; i<str.length(); i++) {
        //storing the character in 2d array's first index
        status[i][0] = String.valueOf(str.charAt(i));

        //checking for invalid character
        if((int)str.charAt(i) < 65 || ((int)str.charAt(i) > 91 &&
(int)str.charAt(i) < 97) || (int)str.charAt(i) > 123) {
            status[i][1] = "not a letter";
        }else if(str.charAt(i) == 'a' || str.charAt(i) == 'e' ||
str.charAt(i) == 'i' || str.charAt(i) == 'o' || str.charAt(i) == 'u') {
            status[i][1] = "vowel"; //checking for vowel
        } else {
            status[i][1] = "consonant"; //checking for
consonant
        }
    }
    //returning the status array
    return status;
}
}

```

- Write a program to trim the leading and trailing spaces from a string using the **charAt()** method

Hint =>

- Create a method to trim the leading and trailing spaces from a string using the `charAt()` method. Inside the method run a couple of loops to trim leading and trailing spaces and determine the starting and ending points with no spaces. Return the start point and end point in an array
- Write a method to create a substring from a string using the `charAt()` method with the string, start, and end index as the parameters
- Write a method to compare two strings using the `charAt()` method and return a boolean result
- The main function calls the user-defined trim and substring methods to get the text after trimming the leading and trailing spaces. Post that use the String built-in method `trim()` to trim spaces and compare the two strings. And finally display the result

```
import java.util.Scanner;
public class StringTrimmer {
    public static void main(String[] args) {

        //creating input object to take inputs
        Scanner input = new Scanner(System.in);

        //taking user inputs for string str
        System.out.print("Enter a String : ");
        String str = input.next();

        //creating variable trimmedString to store the trimmed string
        String trimmedString = trimUsingCharAt(str);

        //printing the trimmed string
        System.out.println("The string without leading and trailing
zeroes is " + trimmedString);

        //taking user inputs for starting and ending indexes
        System.out.print("Enter the starting index : ");
        int start = input.nextInt();
        System.out.print("Enter the ending index : ");
        int end = input.nextInt();

        //creating variable substring to store substring using equals
method
        String substring = substringUsingCharAt(str, start, end);

        //printing the substring
        System.out.println("Substring of " + str + " from index " +
```

```
start + " to " + end + " is " + substring + " using charAt");

    //taking two string for comparison
    System.out.print("Enter another string for comparison : ");
    String anotherStr = input.next();

    //comparing the strings using charAt
    boolean compareByCharAt = compareUsingCharAt(str, anotherStr);

    //printing the comparison
    if(compareByCharAt) {
        System.out.println(str + " and " + anotherStr + " are
same on comparing using charAt.");
    } else {
        System.out.println(str + " and " + anotherStr + " are
different on comparing using charAt.");
    }

    //closing the input
    input.close();
}

//method to trim leading and trailing spaces
public static String trimUsingCharAt(String str) {

    //creating two variables to indicate the starting and ending
point of string
    int start = 0;
    int end = str.length()-1;

    //updating the start to remove leading spaces
    while(str.charAt(start) == ' ') {
        start++;
    }
    //updating the end to remove trailing spaces
    while(str.charAt(end) == ' ') {
        end--;
    }

    //returning the trimmed string
    return substringUsingCharAt(str, start+1, end+2);
}
```

```

    }

    //method to find substring using charAt()
    public static String substringUsingCharAt(String str, int start, int
end) {

        //creating tempArray array to store character array of
character of substring
        char[] tempArray = new char[end - start+1];

        //extracting characters from string
        for(int i=start-1; i<end-1; i++) {
            tempArray[i-start+1] = str.charAt(i);
        }
        //creating string subString to store substring
        String subString = new String(tempArray);

        //returning the substring
        return subString;
    }

    //method to compare two strings using charAt()
    public static boolean compareUsingCharAt(String str1, String str2) {

        //returning false if length are different
        if(str1.length() != str2.length()) {
            return false;
        }

        //returning false if character differ
        for(int i=0; i<str1.length(); i++) {
            if (str1.charAt(i) != str2.charAt(i)) {
                return false;
            }
        }
        //returning true because strings are same
        return true;
    }
}

```

8. Write a program to take user input for the age of all 10 students in a class and check whether the student can vote depending on his/her age is greater or equal to 18.

Hint =>

- a. Create a method to define the random 2 digit age of number of students provided as method parameters and return 1D array of age of n students
- b. Create a method which takes in array of age as a parameter and returns 2D String array of age and boolean true or false to indicate can vote and cannot vote. Inside the method firstly validate the age for a negative number, if a negative cannot vote. For valid age check for age is 18 or above to set true to indicate can vote.
- c. Create a method to display the 2D array in a tabular format.
- d. Finally, the main function takes user inputs, calls the user-defined methods, and displays the result.

```
import java.lang.Math;
public class RandomVoteEligibility {
    public static void main(String []args){

        //generating random ages
        int[] ages = generateAges();

        //creating 2d array to store ages and vote eligibility {age,
true/false}
        String[][] ageVote = voteEligibility(ages);

        //printing the vote eligibility
        printVoteEligibility(ageVote);

    }

    //method to generate random ages
    public static int[] generateAges() {

        //creating array ages to indicate the ages of persons
        int ages[] = new int[10];

        //generating ages of 10 persons
        for(int i=0; i < 10; i++){
            //generating random ages
            ages[i] = (int)(Math.random()*90) + 10;
        }
        //returning ages
    }
}
```



```

        return ages;
    }
    //method to check person can vote or not
    public static String[][] voteEligibility(int[] ages) {

        //creating the 2d array to store ages and vote eligibility
        {age, true/false}
        String[][] ageVote = new String[ages.length][2];

        //checking the vote eligibility
        for(int i=0; i < 10; i++){

            ageVote[i][0] = Integer.toString(ages[i]);

            //validating the age
            if(ages[i]>0) {
                if( ages[i] >= 18 )
                {
                    ageVote[i][1] = "true";
                } else {
                    ageVote[i][1] = "false";
                }
            } else {
                ageVote[i][1] = "Invalid age";
            }
        }
        //returning the ageVote array
        return ageVote;
    }
    //method to print ages and vote in tabular form
    public static void printVoteEligibility(String[][] ageVote) {

        //printing details in tabular form
        System.out.println("Age    Vote Eligibility");
        System.out.println("-----");
        for(int i=0; i<ageVote.length; i++) {
            System.out.println(ageVote[i][0] + "    " +
ageVote[i][1]);
        }
    }
}

```

9. Create a program to take input marks of students in 3 subjects physics, chemistry, and maths. Compute the percentage and then calculate the grade as shown in figure below

Grade	Remarks	Marks
A	(Level 4, above agency-normalized standards)	80% and above
B	(Level 3, at agency-normalized standards)	70-79%
C	(Level 2, below, but approaching agency-normalized standards)	60-69%
D	(Level 1, well below agency-normalized standards)	50-59%
E	(Level 1- , too below agency-normalized standards)	40-49%
R	(Remedial standards)	39% and below

Hint =>

- Write a method to generate random 2-digit scores for Physics, Chemistry and Math (PCM) for the students and return the scores. This method returns a 2D array with PCM scores for all students
- Write a Method to calculate the total, average, and percentages for each student and return a 2D array with the corresponding values. Please ensure to round off the values to 2 Digits using **Math.round()** method
- Write a Method to calculate the grade based on the percentage as shown in the ref table and return a 2D array of students' grade
- Finally write a Method to display the scorecard of all students with their scores, total, average, percentage, and grade in a tabular format.

```
import java.lang.Math;
import java.util.Scanner;
public class StudentGradesString {
    public static void main(String[] args) {

        //creating scanner object to take input
        Scanner input = new Scanner(System.in);

        //creating a variable n to indicate number of students
        System.out.print("Enter number of students : ");
        int n = input.nextInt();

        //creating 2d array to store marks of students in 3 subjects
        int marks[][] = generateMarks(n);
```

```
//creating array to store percentages of students
int[] percentages = calculatePercentages(marks);

//creating array to store total marks of students
int[] total = calculateTotal(marks);

//creating array to store grades of students
char[] grades = calculateGrades(percentages);

//printing the details
System.out.println("Total number of students is " + n);
System.out.println("Student physics chemistry maths total
percentage grade");

System.out.println("-----");

for(int i=0; i<n; i++) {
    System.out.println(i+1 + "      " + marks[i][0] + "
" + marks[i][1] + "      " + marks[i][2] + "      " + total[i] + "      " +
percentages[i] + "      " + grades[i]);
}
//closing the input
input.close();

}
//method to generate random marks
public static int[][] generateMarks(int n) {

    //creating array marks to indicate the marks of students
    int marks[][] = new int[n][3];

    //generating marks of n students
    for(int i=0; i < n; i++){
        //generating random ages
        marks[i][0] = (int)(Math.random()*90) + 10;
        marks[i][1] = (int)(Math.random()*90) + 10;
        marks[i][2] = (int)(Math.random()*90) + 10;

    }
    //returning marks;
```

```

        return marks;
    }
    //method to calculate total marks of students
    public static int[] calculateTotal(int marks[][]) {

        //creating array to store total marks of students
        int[] total = new int[marks.length];

        //calculating total marks
        for(int i=0; i<marks.length; i++) {
            total[i] = Math.round(marks[i][0] + marks[i][1] +
marks[i][2]);
        }
        //returning the total marks
        return total;
    }
    //method to calculate average(percentage) of students
    public static int[] calculatePercentages(int[][] marks) {

        //creating array to store percentages of students
        int[] percentages = new int[marks.length];

        //calculating percentages
        for(int i=0; i<marks.length; i++) {
            percentages[i] = Math.round((marks[i][0] + marks[i][1] +
marks[i][2])/3);
        }
        //returning the percentages
        return percentages;
    }

    //method to calculate grades of students
    public static char[] calculateGrades(int[] percentages) {

        //creating array to store grades of students
        char[] grades = new char[percentages.length];

        for(int i=0; i<percentages.length; i++) {
            if(percentages[i] >= 80) {
                grades[i] = 'A';
            }
        }
    }

```

```
        } else if(percentages[i] >=70) {  
            grades[i] = 'B';  
        } else if(percentages[i] >=60) {  
            grades[i] = 'C';  
        } else if(percentages[i] >=50) {  
            grades[i] = 'D';  
        } else if(percentages[i] >=40) {  
            grades[i] = 'E';  
        } else {  
            grades[i] = 'R';  
        }  
    }  
    //returning the grades  
    return grades;  
}  
}
```